Evaluating Stainless on SV-COMP

Hadi Saleh

18th May 2026

Abstract

Replace this with the actual abstract. Obviously. Suspendisse luctus leo et porta mattis. In semper, nisi et suscipit iaculis, leo urna laoreet lacus, ut laoreet lorem tellus eget dui. Vestibulum eu auctor nisi. Morbi pharetra euismod velit ac mattis. Maecenas tempor vitae augue ut aliquam. Nunc auctor, nibh at imperdiet finibus, ex leo semper lacus, ac vehicula quam nisl condimentum leo.

Contents

1 Introduction **1**
- 1.1 Formal Verification . . . 1
- 1.2 Purpose and Goals . . . 2
- 1.3 Thesis Outline . . . 3

2 Background **5**
- 2.1 Stainless . . . 5
  - 2.1.1 Pure Scala . . . 6
- 2.2 SV-COMP . . . 6
  - 2.2.1 Benchmark Structure . . . 7
  - 2.2.2 Categories . . . 7
  - 2.2.3 Java.Overall . . . 7
  - 2.2.4 Benchmark Infrastructure . . . 8

3 Methodology **9**
- 3.1 Benchmark Selection . . . 9
- 3.2 Translation from Java to Scala . . . 10
  - 3.2.1 Data Types . . . 10
  - 3.2.2 Control Flow . . . 10
  - 3.2.3 Assertions and Properties . . . 11
- 3.3 Specification and Annotation . . . 11
- 3.4 Experimental Setup . . . 13
- 3.5 Data Collection and Analysis . . . 13

4 Design **15**

5 Implementation **17**

6 Evaluation **19**

7 Conclusions **21**

List of Figures

2.1 The summary Stainless provides after verifying the add function in Listing 2.1. 6

3.1 An excluded benchmark utilizing external packages and exception handling. 10
3.2 Translation of the `Ackermann` benchmark, demonstrating the conversion from bounded 32-bit integers to unbounded `BigInt` to avoid overflow semantics. 11
3.3 The translated Scala `sort` method from `InsertionSort-FunSat01`, showing loop invariants and termination measures attached to nested loops. 12
3.4 Translation of the `SatGcd` benchmark demonstrating a non-trivial termination measure: `decreases(y1 + y2)` decreases on every recursive call since each call reduces one argument by the other. 12
3.5 Translation of the `Addition` benchmark demonstrating `BigInt` usage, termination measures, and postconditions. 13

List of Tables

Chapter 1

Introduction

As software becomes increasingly deployed to critical systems in society, everything from medical devices to vehicles to financial infrastructure. As the complexity of such systems advance, the cost of undetected defects drastically increases as well. A major important element of such software then becomes to ensure and verify that they behave correctly and as expected.

A common way to ensure this is to have them tested and verified to make sure that such software and systems do what they are intended to do, therefore making them more reliable and safe to use. Traditional testing methods, where a set of inputs gets run through the program and the outputs get checked against the corresponding expected results. While this is still useful, it would only demonstrate the presence of defects, and not their absence. For programs with large or even infinite input ranges, testing them in such a way would be infeasible.

In response to this, formal verification tools developed to provide mathematical guarantees that a program satisfies its specifications for *all* valid inputs. An example of such a tool is Stainless [5], developed at EPFL for programs written in the Scala programming language. Stainless is a deductive verification framework, meaning that it proves program correctness from formal specifications using mathematical logical reasoning. It verifies the programs functional correctness, termination and absence of runtime errors.

However, such verifiers aren't guaranteed to be flawless and can fail to verify correct programs, timeout, or produce false evaluations. To adress this issue and benchmark verification tools systematically , there is an annual verifier benchmark competition called SV-COMP [2], which maintains a large amount of benchmarks covering a wide range of verification tasks primarly focused on C and Java verifiers and validators.

Although, Stainless appeared promising in different studies (cite studies), it has yet to be evaluated on a large scale benchmark such as SV-COMP. This project addresses the gap by evaluating Stainless on SV-COMP benchmarks through translating the benchmark programs used in SV-COMP on Java verifiers, to equivalent Scala programs and running Stainless on them. The results are then evaluated and compared to the Java verifiers' benchmarks.

1.1 Formal Verification

Formal verification, the underlying concept of this project, is the process of mathematically proving that a program satisfies a set of specifications for all possible inputs.

On the other hand, testing would only be able to demonstrate the presence of bugs on a subset of inputs while formal verification provides guarantees that would hold universally. This makes formal verification particularly valuable in domains where safety is crucial and the cost of undetected defects is high.

There are multiple approaches to formal verification. **Model Checking** is the process of exhaustively exploring a program's state space to check whether a property holds or not. However, it can suffer from state space explosion for large programs [6]. **Static Analysis** approximates the program's behavior to infer properties without having to execute the actual program, meaning that it trades precision for scalability [14]. **Deductive Verification**, the approach most relevant to this project, requires that the developer annotates the program with some formal specifications to describe the expected behaviour of the program, such as preconditions and postconditions on functions, loop invariants, and assertions. The verifier then uses logical reasoning to prove that the implementation satisfies them [11].

SMT (Satisfiability Modulo Theories) solvers are typically relied on by deductive verifiers as they are used to solve the logical formulas that are generated from the program specifications [1]. When a verifier cannot prove a proof obligation, it passes the negation to an SMT solver. The SMT solver would in turn check the negation and if the negation is unsatisfiable, then the property holds. Otherwise, if it is satisfiable, then a counterexample can be produced from the SMT solver and returned to the verifier. SMT solvers are therefore a core part of Stainless's verification process, which is discussed further in Section 2.1.

1.2 Purpose and Goals

The purpose of this project is to evaluate Stainless on SV-COMP benchmarks which would adress the lack of systematic evaluation of Stainless. Since SV-COMP's benchmarks are written in C and Java, it won't be possible to evaluate Stainless on them directly. This project focuses solely on the Java benchmarks, for the reason being that Java and Scala are structurally similar which would make the translation process manageable. A representative subset of those Java benchmarks will be translated to Scala programs that Stainless can process. The subset chosen has a feasible amount of benchmarks for the time period of this project and excludes all the benchmarks that are not translatable to Scala programs that would be compatible with Stainless. Such untranslateable programs include:

- Programs that use Java specific features that are not supported by Stainless
- Programs that use external Java packages that are not available in Stainless

The translated benchmarks are then run on Stainless and the outcomes are compared against both the expected verdicts of the benchmarks which every SV-COMP benchmark has and against the results for the Java verifiers. The evaluation considers the verification outcomes (valid, invalid, timeout or unknown) as well as the verification time and CPU usage.

This project aims to establish a clearer picture of where Stainless stands relative to other verification tools on a standardized set of benchmarks. As well as to identify which types of programs Stainless handles well and which types of programs it struggles with.

1.3 Thesis Outline

Chapter 2

Background

This chapter goes over the foundational concepts and tools necessary to be able to understand this project, the work being done, and the results. I will start by going over formal verification in general, then go over Scala and Stainless and finally go over SV-COMP and its benchmarks and infrastructure.

2.1 Stainless

In this section, I will give an overview of Stainless, the deductive verification framework used in this project. I will describe how it works, its verification process and the subset of Scala it supports, known as Pure Scala.

Stainless [5] is a deductive verifier used to prove correctness properties on Scala programs. Stainless takes Scala programs annotated with specifications such as pre- and postconditions and attempts to prove their correctness using SMT solvers. Properties that Stainless verifies include, termination, absence of runtime errors and functional correctness.

Users annotate the Scala programs with specifications and preconditions such as `require` and postconditions such as `ensuring`. Stainless will then attempt to prove that the specifications hold for all the possible inputs of the functions, using SMT solvers rather than testing. For each condition, Stainless reports one of four verdicts, `valid`, `invalid` (with a counterexample), `timeout` or `unknown`.

The `add` function from Listing 2.1 is a simple example of a Scala function that is annotated with preconditions using `require` and postconditions using `ensuring`. The `require` condition is checked once the function is called whilst the `ensuring` condition is checked once the function has terminated. In this example, if the postcondition is violated for any input values that satisfy the precondition, then Stainless will report an invalid verdict with a counterexample. With this specific example, the postcondition will always be met if the precondition is met, therefore it would be reported as valid by Stainless.

```
1  def add(x: BigInt, y: BigInt): BigInt = {
2    require(x > 0 && y > 0)
3    x + y
4  }.ensuring(res => res > x && res > y)
```

Listing 2.1: A simple annotated Scala function verified by Stainless.

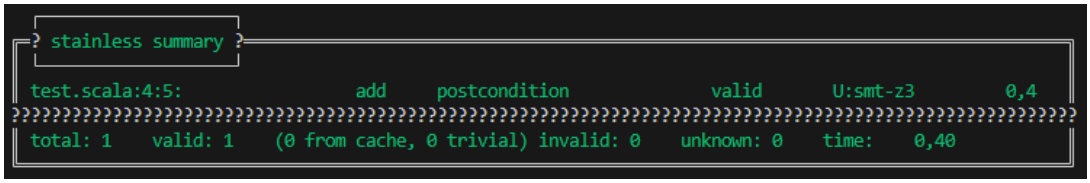


Figure 2.1: The summary Stainless provides after verifying the add function in Listing 2.1.

Stainless is however not compatible with all Scala programs. It only supports a subset of Scala known as Pure Scala.

2.1.1 Pure Scala

Pure Scala is a purely functional subset of Scala that Stainless accepts as input [10]. Understanding it is directly relevant to this project, as it is a key factor in determining which SV-COMP Java benchmarks are translatable into programs that Stainless can process.

Pure Scala is structured around ADTs (algebraic data types), programs are built from `abstract class` hierarchies and immutable `case class` definitions [10]. Pure Scala does support methods, inheritance, and overriding, but restricts classes to immutable fields. However, mutable `var` fields in classes and null references are not supported [13]. Mutable data structures from the Scala standard library are not supported, as Pure Scala enforces a functional, side-effect-free style, which means that functions cannot modify a state outside of their own scope [10].

For numeric types, Pure Scala provides `BigInt` for unbounded integers with no overflow semantics and `Int` as a 32-bit integer with standard overflow semantics. In this project, `BigInt` is preferred over `Int` when translating Java programs that use Java's `int` type, as it avoids overflow cases which complicates the verification process.

Beyond Pure Scala, Stainless also supports an extension that makes it possible to use `var` declarations and `while` loops [9]. Stainless handles them by transforming them into equivalent Pure Scala constructs before verification. This is directly relevant for this project as Java `for` loops are converted into `while` loops in corresponding translated Scala programs. `while` loops can be annotated with loop invariants which Stainless verifies as holding on each iteration of the loop and on termination of the loop.

2.2 SV-COMP

SV-COMP [2] (Competition on Software Verification) is an annual international competition that compares software verification tools on a large set of verification tasks. The most recent edition, SV-COMP 2026, evaluated 61 verifiers and 16 validators for C programs and 11 verifiers and 3 validators for Java programs [4]. All benchmark programs and their specifications are publicly available in the `sv-benchmarks` repository [15], and any researcher may contribute new tasks which fit the competition's quality criteria [2].

2.2.1 Benchmark Structure

Each verification task in SV-COMP consists of a program to be verified, properties which the program is expected to satisfy, and an expected verdict. The expected verdict is either `TRUE` if the program is known to satisfy the property, or `FALSE` if it is known to violate it. This expected verdict is used to determine whether a verifier's output is correct or not.

A verifier participating in SV-COMP produces one of the following outcomes for each task:

- `TRUE`, meaning the verifier concludes that the program satisfies the properties for all possible executions.
- `FALSE`, meaning the verifier concludes that one or all of the properties are violated and typically produces a counterexample.
- `UNKNOWN`, meaning the verifier could not reach a conclusion.
- `Timeout` or `Out Of Memory`, meaning the verifier exhausted its allocated resources without producing a result.

The scoring of each verifier is determined by the correctness of its results. Correct results are awarded points, whilst incorrect results are penalised. A correct `TRUE` or correct `FALSE` result earns points, whereas an incorrect result (a `TRUE` verdict on a task that has an expected verdict of `FALSE`, or vice versa) results in a penalty. This scoring scheme is intentional, as it is considered worse to wrongly assert correctness of a buggy program than to admit inability to decide.

2.2.2 Categories

The benchmark tasks are organised into a hierarchy of categories. At the top level, SV-COMP 2026 has several meta categories for C programs, including `C.ReachSafety`, `C.MemSafety`, `C.Concurrency`, `C.NoOverflows`, `C.Termination`, and `C.SoftwareSystems`, all accumulated under `C.Overall`. There is also a separate meta category for Java programs called `Java.Overall` [4] which is the focus of the translation work in this project.

The C categories contain the vast majority of tasks. For example, in the 2026 edition, `C.Overall` alone contained 36 402 tasks. The Java category is considerably smaller, 1 731 tasks in 2026, covering two properties for Java programs.

2.2.3 Java.Overall

Since this project focuses on translating Java benchmarks to Scala, the `Java.Overall` category is the directly relevant part of SV-COMP. In SV-COMP 2026, it contained two base categories: `Java.valid-assert.Main` and `Java.no-runtime-exception.Main` [4].

The `valid-assert` property requires that all assertions in the program hold for every possible execution. In the Java benchmarks, this is encoded directly through `assert` statements in the source code. The expected verdict is `TRUE` if no assertion can be violated for any possible input, and `FALSE` if an execution reaches a failing assertion. The `no-runtime-exception` property requires that the Java program does not throw a runtime exception under any input.

The Java benchmarks in the repository are drawn from a range of sources, including test suites from established Java verification tools such as `JayHorn` [12] and `JBMC` [7], as well as programs from algorithmic benchmark suites [4]. This variety

means that the tasks range from small programs with a single algorithmic pattern to more complex programs with loops and recursion.

2.2.4 Benchmark Infrastructure

SV-COMP uses BenchExec [3] to execute and measure all verification runs in a controlled and reproducible way. BenchExec enforces resource limits on each run. In 2026 these were 15 GB of memory and 15 minutes of cumulative CPU time on 4 processing units. BenchExec also precisely measures CPU time and memory usage for each task. This execution environment ensures that the results of the different tools and different editions of the competition are comparable.

In this project, Stainless is not run inside the official BenchExec infrastructure, as integrating Stainless into the SV-COMP toolchain is beyond the scope of this project. Instead, the translated benchmarks are run with Stainless directly and the outcomes are compared against the expected SV-COMP verdicts and the results reported by Java verifiers in the competition.

Chapter 3

Methodology

I followed an empirical methodology in which I treated Stainless as the system under study and its verification outcomes on adapted SV-COMP benchmarks served as the data. I first had to choose a subset of the Java benchmarks I was going to evaluate following some parameters (see § 3.1). Then, I translated the selected benchmarks to equivalent Scala programs suitable for Stainless (see § 3.1). While translating the programs I had to add additional annotations and formal specifications to the translated code to make them suitable for Stainless (see § 3.3). I then ran Stainless on the translated benchmarks under the same resource limits as SV-COMP (see § 3.4) and collected the results to then analyze and compare them against the expected SV-COMP verdicts and the results reported by Java verifiers in the competition (see § 3.5).

3.1 Benchmark Selection

The SV-COMP benchmark repository [15] contains a large number of tasks across many categories. As described in § 2.2.3, the `Java.Overall` category in SV-COMP 2026 consists of 1 731 tasks. Evaluating all of these is outside the scope of this project, so I selected a subset of them.

My primary criterion for inclusion in the subset was *translatability*. I included a benchmark only if I could translate it into a Pure Scala program that Stainless can process. I excluded several classes of benchmarks based on these criteria:

- Benchmarks that rely on Java specific features with no direct Scala equivalent.
- Benchmarks that depend on external Java libraries or runtime behaviour that I could not reproduce inside Stainless's supported subset of Scala.

For example, the `RedBlackTree` benchmarks from the `algorithms` category were excluded because they rely heavily on external package imports and exception handling, which aren't directly translatable to Stainless's Pure Scala subset. Figure 3.1 illustrates this untranslatable structure.

Within the remaining translatable benchmarks, I chose a representative subset to cover both property types (`valid-assert` and `no-runtime-exception`) and a range of program structures, including simple programs, programs with loops, and programs with recursion. This variety allows my evaluation to identify patterns in what Stainless handles well and where it encounters difficulties, rather than reporting a single success rate over a homogeneous workload.

```java
import org.sosy_lab.sv_benchmarks.Verifier;
import rbtree.RedBlackTree; // External Java packages which are not supported by Stainless
import rbtree.RedBlackTreeNode; // External Java packages which are not supported by Stainless

public class Main {
  public static void main(String[] args) {
    int N = Verifier.nondetInt();
    Verifier.assume(N > 0);

    try {
      RedBlackTree tree = new RedBlackTree();

      for (int i = 0; i < N; i++) {
        int data = Verifier.nondetInt();
        tree.treeInsert(new RedBlackTreeNode(data));
      }

      int data = Verifier.nondetInt();
      RedBlackTreeNode node = tree.treeSearch(tree.root(), data);
    } catch (Exception e) {
      assert false;
    }
  }
}
```

Figure 3.1: An excluded benchmark utilizing external packages and exception handling.

3.2 Translation from Java to Scala

Since Stainless operates on a functional subset of Scala, the selected Java benchmarks cannot be passed to Stainless directly. I manually translated each benchmark from Java to Scala which is supported by Stainless. I performed the translation manually rather than through an automated tool, because the structural differences between Java and Stainless's supported subset of Scala require some judgement. Specifically, I had to decide how to preserve the program's semantics while conforming to Pure Scala's restrictions, without making any changes to the original meaning of the program or giving Stainless any unfair advantages or disadvantages.

I followed several general principles during the translation process, which I describe below.

3.2.1 Data Types

Java programs primarily use primitive integer types such as `int`. In the translated Scala programs, I systematically replaced Java's `int` with Stainless's `BigInt` type, which represents unbounded mathematical integers and carries no overflow semantics. I made this choice to avoid the complication of modelling Java's 32 bit or 64 bit wrap around arithmetic, which would require modular arithmetic and would complicate the verification process significantly.

An example of this can be seen in the translation of the `Ackermann` benchmark, which is shown in Figure 3.2.

3.2.2 Control Flow

I translated Java's `for` and `while` loops into Scala `while` loops. Stainless allows `while` loops through its imperative extension [9], and converts them into Pure Scala before

Original Java (32 bit int):

```java
public static int ack(int m, int n) {
    if (m == 0) {
      return n + 1;
    }
    if (n == 0) {
      return ack(m - 1, 1);
    }
    return ack(m - 1, ack(m, n - 1));
  }
```

Translated Scala (Unbounded BigInt):

```scala
def ack(m: BigInt, n: BigInt): BigInt = {
    if (m == 0)
      n + 1
    else if (n == 0)
      ack(m - 1, 1)
    else
      ack(m - 1, ack(m, n - 1))
  }
```

Figure 3.2: Translation of the `Ackermann` benchmark, demonstrating the conversion from bounded 32-bit integers to unbounded `BigInt` to avoid overflow semantics.

verifying them. I chose to keep the loop structures exactly as they were in Java to stay faithful to the original benchmarks, rather than rewriting them into recursive functions (which Stainless usually handles more easily).

3.2.3 Assertions and Properties

Benchmarks in SV-COMP often evaluate both properties simultaneously (`valid-assert` and `no-runtime-exception`), and these properties can sometimes have different expected verdicts within the exact same benchmark.

To verify `valid-assert` properties, the original Java benchmarks use `assert` statements. When translating these to Scala, I converted them into Stainless's `assert` expressions or incorporated them directly as function postconditions using Stainless's `ensuring` keyword. Stainless then verifies these statically as proof obligations.

On the other hand, for the `no-runtime-exception` property, no explicit annotations or assertions are required. Stainless automatically verifies the absence of runtime errors such as array out of bounds access and division by zero using its built in verification process [5].

3.3 Specification and Annotation

A key difference between testing and deductive verification is that Stainless must be given enough information about a program's intended behaviour to be able to construct a proof. While translating properties into `assert` or `ensuring` expressions is often enough for simple code, programs with loops or recursion additionally require the user to supply loop invariants or termination measures.

Loop invariants: A loop invariant is a predicate that must hold at the start of each iteration and after the loop terminates. For `while` loops in the translated benchmarks, I provided loop invariants using Stainless's `.invariant()` method attached directly to the loop block. I constructed these invariants by analyzing the logic of the original Java loops. Understanding what the loop was intended to do, as well as knowing whether the benchmark was expected to pass or fail, made it easier to determine what mathematical properties needed to be tracked during verification.

Figure 3.3 shows an example of a translated benchmark with loop invariants. The original benchmark is `InsertionSort-FunSat01`.

Termination measures: Stainless requires termination proofs for all recursive functions to ensure that a non-terminating function cannot trivially satisfy any postcondition [5]. For recursive benchmarks, a termination measure is provided

```scala
def sort(a: Array[Int]): Unit = {
  require(a.length >= 0)
  val N = a.length
  var i = 1
  (while (i < N) {
    decreases(N - i)
    var j = i - 1
    var x = a(i)
    (while ((j >= 0) && (a(j) > x)) {
      decreases(j + 1)
      a(j + 1) = a(j)
      j = j - 1
    }).invariant(j >= -1 && j < i && i < N && a.length == N)
    a(j + 1) = x
    i = i + 1
  }).invariant(i >= 1 && i <= N && a.length == N)
}
```

Figure 3.3: The translated Scala `sort` method from `InsertionSort-FunSat01`, showing loop invariants and termination measures attached to nested loops.

using the `decreases` clause. The measure must be an expression that strictly decreases on every recursive call and is bounded below. Figure 3.4 shows an example from the `SatGcd` benchmark, where the sum `y1 + y2` serves as the measure: each recursive call either reduces `y1` or `y2` by the other, so the sum strictly decreases.

Original Java:

```java
static int gcd(int y1, int y2) {
  if (y1 <= 0 || y2 <= 0)
    return 0;
  if (y1 == y2)
    return y1;
  if (y1 > y2)
    return gcd(y1 - y2, y2);
  return gcd(y1, y2 - y1);
}
```

Translated Scala:

```scala
def gcd(y1: BigInt, y2: BigInt): BigInt = {
  decreases(y1 + y2)
  require(y1 > 0 && y2 > 0)
  if (y1 <= 0 || y2 <= 0)
    BigInt(0)
  else if (y1 == y2)
    y1
  else if (y1 > y2)
    gcd(y1 - y2, y2)
  else
    gcd(y1, y2 - y1)
}.ensuring(res => res >= 1)
```

Figure 3.4: Translation of the `SatGcd` benchmark demonstrating a non-trivial termination measure: `decreases(y1 + y2)` decreases on every recursive call since each call reduces one argument by the other.

Preconditions and postconditions: For benchmarks where the expected verdict is `TRUE`, I added a postcondition using the `.ensuring()` clause to capture the property that the program should satisfy. For benchmarks where the expected verdict is `FALSE`, the goal is to confirm that Stainless produces a counterexample. In these cases, the assertion remains in the translated program, without strengthening the specification, so that Stainless can find the counterexample.

The amount of annotation required varies across the benchmarks. Simple programs often require no annotation beyond the assertion itself, while other benchmarks with loops and complex logic may require more significant annotation effort. Figure 3.5 illustrates a fully annotated recursive benchmark, showing the combination of `BigInt` types, a `decreases` termination measure, and an `.ensuring` postcondition in the `Addition` benchmark.

Original Java:

```java
static int addition(int m, int n) {
  if (n == 0) {
    return m;
  }
  if (n > 0) {
    return addition(m + 1, n - 1);
  }
  return addition(m - 1, n + 1);
}
```

Translated Scala (with annotations):

```scala
def addition(m: BigInt, n: BigInt): BigInt = {
  decreases(if (n >= 0) n else -n)
  if(n == 0){
    m
  } else if(n > 0){
    addition(m + 1, n - 1)
  } else {
    addition(m - 1, n + 1)
  }
}.ensuring(res => res == m + n)
```

Figure 3.5: Translation of the `Addition` benchmark demonstrating `BigInt` usage, termination measures, and postconditions.

3.4 Experimental Setup

I ran all experiments on a single machine with a fixed hardware and software configuration to ensure that timing measurements are reproducible. The relevant configuration is as follows.

- **Stainless version:** I recorded the version of Stainless used in the artifact alongside the translated benchmarks.
- **SMT solver:** I ran Stainless with its default SMT solver configuration, which uses Z3 [8] as the primary backend.
- **Resource limits:** To replicate the official SV-COMP benchmarking environment as closely as possible, each verification run was constrained to a maximum memory limit of 15 GB of RAM and a time limit of 15 minutes (900 seconds).

Unlike the official SV-COMP participants, who are evaluated using the `Benchexec` framework on Linux machines with strict `cgroups` resource isolation, I conducted the experiments for this project locally on a Windows machine using a custom batch script since `Benchexec` does not support Windows or WSL.

3.5 Data Collection and Analysis

For each translated benchmark, I recorded the following data:

- The Stainless verdict: `valid`, `invalid` (with the counterexample if available), `timeout`, or `unknown`.
- The time taken for Stainless to produce the verdict.
- The expected SV-COMP verdict (`TRUE` or `FALSE`).
- Whether the Stainless verdict agrees with the expected verdict.

The primary measure of success is the *correct verdict rate*, which is the fraction of benchmarks for which Stainless's verdict matches the expected SV-COMP verdict. A Stainless `valid` result is taken to correspond to a SV-COMP `TRUE` expected verdict, and a Stainless `invalid` result to a `FALSE` expected verdict.

I used the verification results produced by Java verifiers participating in SV-COMP 2026 as a reference point throughout the analysis.

Chapter 4

Design

Chapter 5

Implementation

Chapter 6

Evaluation

Chapter 7

Conclusions

Bibliography

[1] Clark Barrett, Pascal Fontaine, and Cesare Tinelli. The Satisfiability Modulo Theories Library (SMT-LIB), 2016. https://smt-lib.org/.

[2] Dirk Beyer. Competition on Software Verification (SV-COMP), 2026. https://sv-comp.sosy-lab.org/.

[3] Dirk Beyer, Stefan Löwe, and Philipp Wendler. Reliable benchmarking: Requirements and solutions. *International Journal on Software Tools for Technology Transfer*, 21(1):1–29, 2019. doi:10.1007/s10009-017-0469-y.

[4] Dirk Beyer and Jan Strejček. Evaluating software verifiers for C, Java, and SV-LIB. In *Tools and Algorithms for the Construction and Analysis of Systems*, volume 16506 of *Lecture Notes in Computer Science*. Springer, 2026. doi:10.1007/978-3-032-22749-2_23.

[5] Régis Blanc, Viktor Kuncak, Etienne Kneuss, and Philippe Suter. An Overview of the Leon Verification System. In *Proceedings of the 3rd Workshop on Scala (SCALA)*. ACM, 2013. doi:10.1145/2489837.2489840.

[6] Edmund M. Clarke, Orna Grumberg, and Doron A. Peled. *Model Checking*. MIT Press, 1999.

[7] Lucas C. Cordeiro, Pascal Kesseli, Daniel Kroening, Peter Schrammel, and Marek Trtík. JBMC: A bounded model checking tool for verifying Java bytecode. In *Computer Aided Verification (CAV)*, volume 10981 of *Lecture Notes in Computer Science*, pages 183–190, 2018. doi:10.1007/978-3-319-96145-3_10.

[8] Leonardo de Moura and Nikolaj Bjørner. Z3: An efficient SMT solver. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, volume 4963 of *Lecture Notes in Computer Science*, pages 337–340, 2008. doi:10.1007/978-3-540-78800-3_24.

[9] EPFL LARA. Imperative and other effects — Stainless documentation. https://epfl-lara.github.io/stainless/imperative.html, 2024. Accessed: 2025.

[10] EPFL LARA. Pure Scala Stainless Documentation. https://epfl-lara.github.io/stainless/purescala.html, 2024. Accessed: 2025.

[11] Jean-Christophe Filliâtre and Andrei Paskevich. Why3: Where programs meet provers. In *Programming Languages and Systems (ESOP)*, volume 7792 of *Lecture Notes in Computer Science*, pages 125–128, 2013. doi:10.1007/978-3-642-37036-6_8.

[12] Temesghen Kahsai, Philipp Rümmer, Huascar Sanchez, and Martin Schäf. JayHorn: A framework for verifying Java programs. In *Computer Aided Verification (CAV)*, volume 9780 of *Lecture Notes in Computer Science*, pages 352–358, 2016. doi:10.1007/978-3-319-41540-6_19.

[13] Viktor Kuncak and Jad Hamza. Stainless verification system tutorial. In *Formal Methods in Computer Aided Design, FMCAD 2021, New Haven, CT, USA, October 19-22, 2021*, pages 2–7. IEEE, 2021. doi:10.34727/2021/ISBN.978-3-85448-046-4_2.

[14] Flemming Nielson, Hanne Riis Nielson, and Chris Hankin. *Principles of Program Analysis*. Springer, 1999. doi:10.1007/978-3-662-03811-6.

[15] SoSy-Lab. SV-Benchmarks: Software Verification Benchmarks, 2026. https://gitlab.com/sosy-lab/benchmarking/sv-benchmarks.